CirculAR — Modelo relacional inicial

1. Objetivo

Definir el modelo relacional inicial de CirculAR sobre PostgreSQL, alineado con:

- blueprint de producto
- definiciones oficiales
- dominio oficial
- estados e invariantes
- contratos API v1
- arquitectura backend v1

Este documento fija:

- tablas principales
- relaciones
- claves primarias y foráneas
- enums base
- constraints
- índices
- decisiones de soft delete
- reglas anti-inconsistencia
- base para migraciones v1

---

2. Principios de modelado

2.1 La base debe reforzar invariantes reales

No toda regla vive solo en código. La base debe ayudar a impedir estados imposibles o inconsistencias frecuentes.

2.2 El modelo es transaccional primero

La prioridad del modelo relacional v1 es sostener bien:

- publicación
- disponibilidad
- interacciones
- reserva
- match
- conversación

- moderación
- reputación
- notificaciones

2.3 Read models no se materializan todavía salvo necesidad

En v1, las vistas de UI se construyen con queries y assemblers. No hace falta crear tablas proyección para todo desde el primer día. La verdad operativa vive en tablas transaccionales.

2.4 Soft delete selectivo

Se usará soft delete donde tenga sentido operativo, no como reflejo automático en todo el sistema.

2.5 Catálogos cerrados y enums explícitos

Los estados críticos y muchos atributos del negocio deben modelarse como enums controlados.

---

3. Convenciones generales

3.1 Identificadores

Recomendación:

- PK técnica `uuid` en todas las tablas principales
- opcionalmente exponer IDs públicos con prefijo a nivel API en una capa aparte

Ejemplo interno:

- `id uuid primary key`

Ejemplo externo posible:

- `lst_xxx`
- `usr_xxx`
- `pi_xxx`

No es obligatorio persistir el prefijo visible como PK real.

3.2 Fechas comunes

Siempre que aplique:

- `created_at timestamptz not null`

- `updated_at timestamptz not null`

Y según tabla:

- `deleted_at timestamptz null`
- `archived_at timestamptz null`
- `resolved_at timestamptz null`
- `expires_at timestamptz null`

3.3 Auditoría mínima de actor

En tablas de acciones o eventos críticos conviene incluir:

- `created_by_user_id`
- `resolved_by_user_id`
- `last_transition_by`

según corresponda.

---

4. Enums iniciales oficiales

4.1 ListingState

- `DRAFT`
- `IN_REVIEW`
- `OBSERVED`
- `PUBLISHED`
- `PAUSED`
- `RESERVED`
- `CLOSED`
- `REJECTED`
- `ARCHIVED`

4.2 PurchaseIntentState

- `ACTIVE`
- `ACCEPTED`
- `REJECTED`
- `CANCELLED`
- `EXPIRED`
- `CLOSED`

4.3 TradeProposalState

- `ACTIVE`
- `ACCEPTED`
- `REJECTED`
- `CANCELLED`
- `EXPIRED`
- `CLOSED`

4.4 MatchSessionState

- `OPEN`
- `ACTIVE`
- `COMPLETED`
- `FAILED`
- `EXPIRED`
- `CANCELLED`
- `CLOSED`

4.5 ConversationThreadState

- `OPEN`
- `RESTRICTED`
- `CLOSED`
- `ARCHIVED`

4.6 ModerationReviewState

- `PENDING`
- `APPROVED`
- `OBSERVED`
- `REJECTED`
- `SUPERSEDED`

4.7 ImageAuditStatus

- `PENDING`
- `APPROVED`
- `OBSERVED`
- `REJECTED`
- `ERROR`

4.8 NotificationState

- `UNREAD`
- `READ`
- `ARCHIVED`

4.9 InteractionType

- `PURCHASE_INTENT`
- `TRADE_PROPOSAL`

4.10 MatchType

- `PURCHASE`
- `TRADE`

4.11 MessageType

- `TEXT`
- `QUICK_ACTION`
- `SYSTEM`

4.12 QuickActionCode

Catálogo inicial sugerido:

- `PROPOSE_MEETING`
- `SHARE_AVAILABILITY`
- `CONFIRM_INTEREST_STILL_ACTIVE`
- `MARK_ON_THE_WAY`
- `REQUEST_CONTACT`

4.13 ListingMode

No como estado único, sino como configuración comercial derivable. Aun así, si querés persistirlo explícitamente para simplificar reads:

- `PURCHASE_ONLY`
- `TRADE_ONLY`
- `BOTH`

4.14 ModerationReasonCode

Catálogo inicial sugerido:

- `PHOTO_BLURRY`
- `PHOTO_TOO_DARK`

- `PHOTO_TOO_FEW`
- `PHOTO_DUPLICATED`
- `PHOTO_CONTENT_NOT_ALLOWED`
- `NOT_A_GARMENT`
- `CATEGORY_MISMATCH`
- `MISSING_REQUIRED_DATA`
- `LOW_LISTING_QUALITY`

4.15 ReportReasonCode

- `INAPPROPRIATE_CONTENT`
- `FRAUD_SUSPICION`
- `HARASSMENT`
- `SPAM`
- `OTHER`

---

5. Tablas núcleo

5.1 users

Representa identidad base del usuario.

Campos sugeridos:

- `id uuid pk`
- `email varchar(320) not null unique`
- `password_hash varchar not null`
- `phone_e164 varchar(24) not null unique`
- `is_phone_verified boolean not null default false`
- `status varchar not null default 'ACTIVE'`
- `created_at timestamptz not null`
- `updated_at timestamptz not null`

Índices:

- unique `users_email_uq`
- unique `users_phone_uq`

Notas:

- `status` puede empezar simple y luego volverse enum si aparecen estados más formales como `BLOCKED` o `SUSPENDED`.

5.2 sessions

Sesiones y refresh tokens revocables.

Campos sugeridos:

- `id uuid pk`
- `user_id uuid not null fk -> users.id`
- `refresh_token_hash varchar not null`
- `device_info jsonb null`
- `ip_address inet null`
- `user_agent text null`
- `expires_at timestamptz not null`
- `revoked_at timestamptz null`
- `created_at timestamptz not null`
- `updated_at timestamptz not null`

Índices:

- `idx_sessions_user_id`
- `idx_sessions_expires_at`

5.3 public_profiles

Perfil público resumido.

Campos sugeridos:

- `id uuid pk`
- `user_id uuid not null unique fk -> users.id`
- `first_name varchar(80) not null`
- `last_name varchar(80) null`
- `instagram_handle varchar(64) null`
- `city varchar(120) not null`
- `zone varchar(120) null`
- `bio varchar(280) null`
- `avatar_url text null`
- `created_at timestamptz not null`
- `updated_at timestamptz not null`

Índices:

- unique `public_profiles_user_id_uq`
- `idx_public_profiles_city_zone`

5.4 trust_profiles

Señales base no derivadas y algunas flags operativas.

Campos sugeridos:

- `id uuid pk`
- `user_id uuid not null unique fk -> users.id`
- `has_instagram boolean not null default false`
- `instagram_verified boolean not null default false`
- `manual_review_required boolean not null default false`
- `restriction_flags jsonb not null default '{}'`
- `created_at timestamptz not null`
- `updated_at timestamptz not null`

5.5 reach_zones

Zonas donde el usuario está dispuesto a concretar cambios o compras.

Campos sugeridos:

- `id uuid pk`
- `user_id uuid not null fk -> users.id`
- `city varchar(120) not null`
- `zone varchar(120) not null`
- `created_at timestamptz not null`

Constraints:

- unique `(user_id, city, zone)`

Índices:

- `idx_reach_zones_user_id`
- `idx_reach_zones_city_zone`

5.6 reputation_profiles

Métricas agregadas visibles del usuario.

Campos sugeridos:

- `id uuid pk`
- `user_id uuid not null unique fk -> users.id`
- `completed_transactions_count integer not null default 0`
- `successful_transactions_count integer not null default 0`
- `failed_transactions_count integer not null default 0`
- `cancelled_transactions_count integer not null default 0`
- `success_rate numeric(5,4) not null default 0`
- `avg_response_time_hours numeric(10,2) null`
- `last_recomputed_at timestamptz null`
- `created_at timestamptz not null`
- `updated_at timestamptz not null`

Notas:

- `success_rate` puede ser derivada y persistida como snapshot por performance/simplicidad.

---

6. Listings y catálogo

6.1 garments

Datos propios de la prenda física.

Campos sugeridos:

- `id uuid pk`
- `owner_user_id uuid not null fk -> users.id`
- `category varchar(80) not null`
- `subcategory varchar(80) null`
- `size varchar(32) not null`
- `condition varchar(32) not null`
- `brand varchar(120) null`
- `color varchar(80) null`
- `material varchar(120) null`
- `created_at timestamptz not null`
- `updated_at timestamptz not null`

Índices:

- `idx_garments_owner_user_id`
- `idx_garments_category_size`

Nota:

- Separar `garments` de `listings` mantiene más limpio el agregado y deja mejor preparada una futura reutilización o histórico.

6.2 listings

Raíz comercial de publicación.

Campos sugeridos:

- `id uuid pk`
- `owner_user_id uuid not null fk -> users.id`
- `garment_id uuid not null unique fk -> garments.id`
- `state listing_state not null`
- `description text null`
- `allows_purchase boolean not null default false`
- `allows_trade boolean not null default false`
- `price_amount integer null`
- `currency_code char(3) not null default 'ARS'`
- `city varchar(120) not null`
- `zone varchar(120) null`
- `quality_score integer null`
- `dominant_photo_id uuid null`
- `active_moderation_review_id uuid null`
- `reservation_expires_at timestamptz null`
- `published_at timestamptz null`
- `closed_at timestamptz null`
- `archived_at timestamptz null`
- `deleted_at timestamptz null`
- `created_at timestamptz not null`
- `updated_at timestamptz not null`

Constraints recomendadas:

- check: `allows_purchase = true or allows_trade = true`
- check: `price_amount is null or price_amount >= 0`

- check: si `allows_purchase = false`, `price_amount` puede ser null
- check: si `state = 'RESERVED'`, `reservation_expires_at is not null`
- check: si `state = 'CLOSED'`, `closed_at is not null`
- check: si `state = 'ARCHIVED'`, `archived_at is not null`

Índices:

- `idx_listings_owner_user_id`
- `idx_listings_state`
- `idx_listings_city_zone`
- `idx_listings_published_feed` sobre `(state, city, zone, published_at desc)`
- `idx_listings_allows_purchase_trade`
- parcial para feed: `(published_at desc)` where `state = 'PUBLISHED' and deleted_at is null`

Regla importante:

- `dominant_photo_id` debe referenciar una foto del mismo listing. Eso puede reforzarse en aplicación o con constraint más compleja si se modela clave compuesta.

6.3 listing_photos

Fotos de la publicación.

Campos sugeridos:

- `id uuid pk`
- `listing_id uuid not null fk -> listings.id`
- `object_key text not null`
- `public_url text not null`
- `mime_type varchar(120) not null`
- `size_bytes integer not null`
- `width integer not null`
- `height integer not null`
- `checksum_sha256 varchar(64) null`
- `position integer not null`
- `audit_status image_audit_status not null default 'PENDING'`
- `uploaded_at timestamptz not null`
- `created_at timestamptz not null`
- `updated_at timestamptz not null`

Constraints:

- unique `(listing_id, position)`
- check `position >= 1`
- check `size_bytes > 0`
- check `width > 0 and height > 0`

Índices:

- `idx_listing_photos_listing_id`
- `idx_listing_photos_audit_status`

6.4 listing_trade_preferences

Preferencias del dueño para permuta.

Campos sugeridos:

- `id uuid pk`
- `listing_id uuid not null unique fk -> listings.id`
- `desired_categories jsonb not null default '[]'`
- `desired_sizes jsonb not null default '[]'`
- `notes varchar(280) null`
- `created_at timestamptz not null`
- `updated_at timestamptz not null`

Notas:

- Para MVP, JSONB es aceptable aquí. Si en el futuro se vuelve criterio central de matching complejo, puede normalizarse.

6.5 saved_listings

Guardados personales.

Campos sugeridos:

- `id uuid pk`
- `user_id uuid not null fk -> users.id`
- `listing_id uuid not null fk -> listings.id`
- `created_at timestamptz not null`

Constraints:

- unique `(user_id, listing_id)`

Índices:

- `idx_saved_listings_user_id_created_at`
- `idx_saved_listings_listing_id`

---

7. Moderación

7.1 moderation_reviews

Revisión operativa de una publicación.

Campos sugeridos:

- `id uuid pk`
- `listing_id uuid not null fk -> listings.id`
- `state moderation_review_state not null`
- `reasons jsonb not null default '[]'`
- `provider_summary jsonb null`
- `review_version integer not null default 1`
- `started_at timestamptz not null`
- `resolved_at timestamptz null`
- `superseded_at timestamptz null`
- `created_by_system boolean not null default true`
- `resolved_by_user_id uuid null fk -> users.id`
- `created_at timestamptz not null`
- `updated_at timestamptz not null`

Constraints:

- check: si `state in ('APPROVED','OBSERVED','REJECTED','SUPERSEDED')`, algunas fechas deben existir según política

Índices:

- `idx_moderation_reviews_listing_id`
- `idx_moderation_reviews_state`
- parcial recomendado: `(listing_id, created_at desc)`

Regla importante:

- debe existir a lo sumo una review vigente no superseded relevante por listing. Esto puede reforzarse con un índice parcial, por ejemplo sobre `listing_id` where `state in ('PENDING','APPROVED','OBSERVED')`, según política exacta.

7.2 image_audits

Auditoría por imagen.

Campos sugeridos:

- `id uuid pk`
- `listing_photo_id uuid not null fk -> listing_photos.id`
- `status image_audit_status not null`
- `reasons jsonb not null default '[]'`
- `provider_name varchar(80) null`
- `provider_payload jsonb null`
- `audited_at timestamptz null`
- `created_at timestamptz not null`
- `updated_at timestamptz not null`

Índices:

- `idx_image_audits_listing_photo_id`
- `idx_image_audits_status`

7.3 user_reports

Reportes básicos de listing o conducta.

Campos sugeridos:

- `id uuid pk`
- `reporter_user_id uuid not null fk -> users.id`
- `target_user_id uuid null fk -> users.id`
- `target_listing_id uuid null fk -> listings.id`
- `reason_code report_reason_code not null`
- `details text null`
- `state varchar not null default 'OPEN'`
- `resolved_at timestamptz null`

- `resolved_by_user_id uuid null fk -> users.id`
- `created_at timestamptz not null`
- `updated_at timestamptz not null`

Constraints:

- check: al menos uno entre `target_user_id` y `target_listing_id` no puede ser null

Índices:

- `idx_user_reports_reporter_user_id`
- `idx_user_reports_target_listing_id`
- `idx_user_reports_target_user_id`
- `idx_user_reports_state`

---

8. Interacciones

8.1 purchase_intents

Intención estructurada de compra.

Campos sugeridos:

- `id uuid pk`
- `listing_id uuid not null fk -> listings.id`
- `buyer_user_id uuid not null fk -> users.id`
- `listing_owner_user_id uuid not null fk -> users.id`
- `state purchase_intent_state not null`
- `source varchar(40) null`
- `expires_at timestamptz null`
- `accepted_at timestamptz null`
- `rejected_at timestamptz null`
- `cancelled_at timestamptz null`
- `closed_at timestamptz null`
- `resolved_by_user_id uuid null fk -> users.id`
- `created_at timestamptz not null`
- `updated_at timestamptz not null`

Constraints:

- check `buyer_user_id <> listing_owner_user_id`

Índices:

- `idx_purchase_intents_listing_id`
- `idx_purchase_intents_buyer_user_id`
- `idx_purchase_intents_listing_owner_user_id`
- `idx_purchase_intents_state`

Restricción crítica:

- unique parcial sobre `(listing_id, buyer_user_id)` where `state = 'ACTIVE'`

Esto refuerza el invariante de una sola intención activa por usuario y listing. fileciteturn1file3

8.2 trade_proposals

Propuesta estructurada de permuta.

Campos sugeridos:

- `id uuid pk`
- `target_listing_id uuid not null fk -> listings.id`
- `proposer_user_id uuid not null fk -> users.id`
- `target_listing_owner_user_id uuid not null fk -> users.id`
- `state trade_proposal_state not null`
- `source varchar(40) null`
- `expires_at timestamptz null`
- `accepted_at timestamptz null`
- `rejected_at timestamptz null`
- `cancelled_at timestamptz null`
- `closed_at timestamptz null`
- `resolved_by_user_id uuid null fk -> users.id`
- `created_at timestamptz not null`
- `updated_at timestamptz not null`

Constraints:

- check `proposer_user_id <> target_listing_owner_user_id`

Índices:

- `idx_trade_proposals_target_listing_id`
- `idx_trade_proposals_proposer_user_id`

- `idx_trade_proposals_target_listing_owner_user_id`
- `idx_trade_proposals_state`

Restricción crítica:

- unique parcial sobre `(target_listing_id, proposer_user_id)` where `state = 'ACTIVE'`

8.3 trade_proposal_items

Listings ofrecidos por el proponente dentro de una propuesta.

Campos sugeridos:

- `id uuid pk`
- `trade_proposal_id uuid not null fk -> trade_proposals.id`
- `proposed_listing_id uuid not null fk -> listings.id`
- `created_at timestamptz not null`

Constraints:

- unique `(trade_proposal_id, proposed_listing_id)`

Índices:

- `idx_trade_proposal_items_trade_proposal_id`
- `idx_trade_proposal_items_proposed_listing_id`

Reglas de aplicación importantes:

- el `proposed_listing_id` debe pertenecer al `proposer_user_id`
- debe estar vigente y no comprometido incompatiblemente
- debe existir al menos uno por proposal antes de activarla

8.4 proposed_listing_commitments

Tabla operativa recomendada para reforzar la regla más delicada de permuta: evitar comprometer simultáneamente una misma prenda ofrecida en acuerdos incompatibles.

Campos sugeridos:

- `id uuid pk`
- `proposed_listing_id uuid not null fk -> listings.id`

- `trade_proposal_id uuid null fk -> trade_proposals.id`
- `match_session_id uuid null`
- `state varchar not null`
- `created_at timestamptz not null`
- `released_at timestamptz null`

Estados sugeridos:

- `RESERVED_FOR_PROPOSAL`
- `COMMITTED_TO_MATCH`
- `RELEASED`

Restricción recomendada:

- unique parcial sobre `(proposed_listing_id)` where `state in ('RESERVED_FOR_PROPOSAL', 'COMMITTED_TO_MATCH')`

Comentario honesto:

- esta tabla no es estrictamente obligatoria para un MVP muy chico, pero sí es una muy buena decisión si querés robustez real desde temprano en permutas 1:N. Sin algo así, esta regla queda demasiado confiada al código y es más fácil romperla bajo concurrencia. Esto está directamente alineado con la decisión de impedir comprometer un ProposedItem en múltiples MatchSession incompatibles. fileciteturn1file1 fileciteturn1file3

---

9. Match y conversación

9.1 match_sessions

Contexto operativo post-aceptación.

Campos sugeridos:

- `id uuid pk`
- `type match_type not null`
- `state match_session_state not null`
- `listing_id uuid not null fk -> listings.id`
- `origin_purchase_intent_id uuid null fk -> purchase_intents.id`
- `origin_trade_proposal_id uuid null fk -> trade_proposals.id`
- `owner_user_id uuid not null fk -> users.id`
- `counterparty_user_id uuid not null fk -> users.id`
- `expires_at timestamptz not null`

- `completed_at timestamptz null`
- `failed_at timestamptz null`
- `cancelled_at timestamptz null`
- `closed_at timestamptz null`
- `success_confirmed_by_owner_at timestamptz null`
- `success_confirmed_by_counterparty_at timestamptz null`
- `created_at timestamptz not null`
- `updated_at timestamptz not null`

Constraints:

- check `owner_user_id <> counterparty_user_id`
- check: exactamente uno de `origin_purchase_intent_id` u `origin_trade_proposal_id` debe ser no null
- check: si `type = 'PURCHASE'`, entonces `origin_purchase_intent_id is not null`
- check: si `type = 'TRADE'`, entonces `origin_trade_proposal_id is not null`

Índices:

- `idx_match_sessions_listing_id`
- `idx_match_sessions_owner_user_id`
- `idx_match_sessions_counterparty_user_id`
- `idx_match_sessions_state`
- `idx_match_sessions_expires_at`

Restricción crítica:

- unique parcial sobre `(listing_id)` where `state in ('OPEN','ACTIVE')`

Esto refuerza la regla de una sola MatchSession activa por listing. fileciteturn1file3

9.2 conversation_threads

Thread contextual asociado a una match.

Campos sugeridos:

- `id uuid pk`
- `match_session_id uuid not null unique fk -> match_sessions.id`
- `state conversation_thread_state not null`
- `restricted_at timestamptz null`

- `closed_at timestamptz null`
- `archived_at timestamptz null`
- `created_at timestamptz not null`
- `updated_at timestamptz not null`

Restricción:

- unique `(match_session_id)`

Comentario:

- mantener 1 thread por match simplifica muchísimo el modelo y encaja perfecto con el dominio ya definido. fileciteturn1file1

9.3 conversation_messages

Mensajes del thread.

Campos sugeridos:

- `id uuid pk`
- `conversation_thread_id uuid not null fk -> conversation_threads.id`
- `sender_user_id uuid null fk -> users.id`
- `message_type message_type not null`
- `text_body text null`
- `quick_action_code quick_action_code null`
- `metadata jsonb not null default '{}'`
- `created_at timestamptz not null`

Constraints:

- check: si `message_type = 'TEXT'`, `text_body is not null`
- check: si `message_type = 'QUICK_ACTION'`, `quick_action_code is not null`
- check: si `message_type = 'SYSTEM'`, `sender_user_id` puede ser null

Índices:

- `idx_conversation_messages_thread_id_created_at`
- `idx_conversation_messages_sender_user_id`

---

10. Reputación

10.1 reputation_entries

Registro derivado de interacción concluida.

Campos sugeridos:

- `id uuid pk`
- `user_id uuid not null fk -> users.id`
- `match_session_id uuid not null fk -> match_sessions.id`
- `entry_type varchar(40) not null`
- `is_success boolean not null`
- `response_time_hours numeric(10,2) null`
- `was_cancelled boolean not null default false`
- `notes jsonb not null default '{}'`
- `created_at timestamptz not null`

Índices:

- `idx_reputation_entries_user_id`
- `idx_reputation_entries_match_session_id`

Restricción sugerida:

- unique `(user_id, match_session_id, entry_type)`

---

11. Notificaciones

11.1 notifications

Notificaciones persistidas.

Campos sugeridos:

- `id uuid pk`
- `user_id uuid not null fk -> users.id`
- `type varchar(80) not null`
- `state notification_state not null default 'UNREAD'`
- `title varchar(160) not null`
- `body text null`
- `payload jsonb not null default '{}'`

- `read_at timestamptz null`
- `archived_at timestamptz null`
- `created_at timestamptz not null`

Índices:

- `idx_notifications_user_id_created_at`
- `idx_notifications_user_id_state`

---

12. Outbox y eventos operativos

12.1 domain_events_outbox

Base para entrega confiable de eventos.

Campos sugeridos:

- `id uuid pk`
- `aggregate_type varchar(80) not null`
- `aggregate_id uuid not null`
- `event_type varchar(120) not null`
- `payload jsonb not null`
- `status varchar(32) not null default 'PENDING'`
- `available_at timestamptz not null default now()`
- `processed_at timestamptz null`
- `attempt_count integer not null default 0`
- `last_error text null`
- `created_at timestamptz not null`

Índices:

- `idx_outbox_status_available_at`
- `idx_outbox_aggregate_type_id`

Estados sugeridos:

- `PENDING`
- `PROCESSING`
- `PROCESSED`
- `FAILED`

---

13. Relaciones principales

Resumen de relaciones:

- `users 1 - 1 public_profiles`
- `users 1 - 1 trust_profiles`
- `users 1 - 1 reputation_profiles`
- `users 1 - n reach_zones`
- `users 1 - n garments`
- `users 1 - n listings`
- `listings 1 - 1 garments`
- `listings 1 - n listing_photos`
- `listings 1 - 0..1 listing_trade_preferences`
- `users n - n listings` vía `saved_listings`
- `listings 1 - n purchase_intents`
- `listings 1 - n trade_proposals`
- `trade_proposals 1 - n trade_proposal_items`
- `listings 1 - n moderation_reviews`
- `listing_photos 1 - n image_audits`
- `listings 1 - n user_reports`
- `listings 1 - n match_sessions`
- `match_sessions 1 - 1 conversation_threads`
- `conversation_threads 1 - n conversation_messages`
- `match_sessions 1 - n reputation_entries`
- `users 1 - n notifications`

---

14. Reglas anti-inconsistencia que deben reforzarse en base

14.1 Una PurchaseIntent activa por usuario y listing

Resolver con índice unique parcial:

- `(listing_id, buyer_user_id)` where `state = 'ACTIVE'`

14.2 Una TradeProposal activa por usuario y target listing

Resolver con índice unique parcial:

- `(target_listing_id, proposer_user_id)` where `state = 'ACTIVE'`

14.3 Una MatchSession activa por listing

Resolver con índice unique parcial:

- `(listing_id)` where `state in ('OPEN','ACTIVE')`

14.4 Proposed item no comprometido incompatiblemente

Resolver idealmente con `proposed_listing_commitments` + unique parcial sobre `proposed_listing_id` en estados activos.

14.5 Listing reservado no ambiguo

Reforzar con:

- check sobre `reservation_expires_at`
- transacciones con lock sobre listing al aceptar interacción

14.6 Moderation review vigente única según política

Resolver con política clara + índice parcial si decidís que solo puede existir una review `PENDING` o una vigente no terminal por listing.

---

15. Índices de producto más importantes

Además de PK/FK y unique obvios, los más importantes para UX/performance inicial son:

Feed

- `listings(state, city, zone, published_at desc)`
- `garments(category, size)`
- parcial de listings publicados no archivados

Inbox de interacciones

- `purchase_intents(listing_owner_user_id, state, created_at desc)`

- `trade_proposals(target_listing_owner_user_id, state, created_at desc)`

Mis listings

- `listings(owner_user_id, state, updated_at desc)`

Matches

- `match_sessions(owner_user_id, state, created_at desc)`
- `match_sessions(counterparty_user_id, state, created_at desc)`

Notificaciones

- `notifications(user_id, state, created_at desc)`

Jobs automáticos

- `listings(state, reservation_expires_at)` where `state = 'RESERVED'`
- `match_sessions(state, expires_at)` where `state in ('OPEN','ACTIVE')`
- `domain_events_outbox(status, available_at)`

---

16. Soft delete y archivado

16.1 listings

Mantener:

- `state = ARCHIVED`
- `archived_at`
- `deleted_at` opcional si querés conservar una distinción entre archivo funcional y borrado lógico técnico

Mi recomendación:

- usar principalmente `state + archived_at`
- reservar `deleted_at` para casos excepcionales internos

16.2 conversation_threads

No hace falta soft delete clásico. Mejor:

- `state = ARCHIVED`
- `archived_at`

16.3 notifications

Conviene `state = ARCHIVED` y `archived_at` si luego querés limpieza o retención parcial.

---

17. Decisiones específicas importantes

17.1 Guardar owner redundante en tablas de interacción

En `purchase_intents` y `trade_proposals` conviene persistir también el owner del listing al momento de creación:

- simplifica queries de inbox
- desacopla lecturas frecuentes
- evita joins innecesarios en superficies P0

17.2 1 listing = 1 garment

Se mantiene formalmente en el modelo con `garment_id unique` en `listings`, alineado con la decisión ya fijada de una publicación por prenda física. fileciteturn1file1turn1file2

17.3 Permuta 1 a N sin dinero

El modelo lo soporta naturalmente con `trade_proposal_items` de cardinalidad 1..N, sin columna de compensación monetaria en MVP. Esto sigue lo definido para permuta estructurada 1 a N sin agregar dinero. fileciteturn1file2

17.4 Un chat por match, no chat global

Se refuerza con `conversation_threads.match_session_id unique`, totalmente alineado con que la conversación solo existe dentro de una MatchSession. fileciteturn1file0turn1file1turn1file3

---

18. Orden sugerido de migraciones iniciales

M1

- users

- sessions
- public_profiles
- trust_profiles
- reach_zones
- reputation_profiles

M2

- garments
- listings
- listing_photos
- listing_trade_preferences
- saved_listings

M3

- moderation_reviews
- image_audits
- user_reports

M4

- purchase_intents
- trade_proposals
- trade_proposal_items
- proposed_listing_commitments

M5

- match_sessions
- conversation_threads
- conversation_messages

M6

- reputation_entries
- notifications
- domain_events_outbox

M7

- índices parciales avanzados
- constraints adicionales
- backfills o data fixes si hicieran falta

---

19. Qué no conviene hacer todavía

- tablas proyección para cada pantalla
- modelado hipernormalizado de categorías/talles si todavía no tenés catálogo oficial cerrado
- event store completo
- versionado interno de todas las entidades
- particionado prematuro
- tablas analíticas separadas desde el día 1

---

20. Riesgos que este modelo evita

- múltiples verdades de disponibilidad
- más de una interacción activa igual por usuario y listing
- más de una match activa para el mismo listing
- propuesta de permuta sin prendas ofrecidas claras
- conversación fuera de contexto
- moderación opaca sin trazabilidad
- notificaciones no persistidas
- pérdida de eventos al salir del commit

---

21. Resultado esperado de esta etapa

Con este modelo relacional inicial, CirculAR ya tiene una base suficiente para:

1. escribir migraciones reales
2. definir entidades ORM con criterio
3. implementar repositories por módulo
4. construir el scaffold técnico NestJS v1
5. arrancar la implementación por fases sin ambigüedad estructural

---

22. Próxima etapa recomendada

La siguiente pieza correcta es:

1. **Catálogo de enums y constantes oficiales**
2. **Scaffold técnico NestJS v1**
3. **Roadmap de implementación por módulos con criterios de aceptación**